```
{

    return  (1 & ((my_word[pos/32]  >> (pos
% 32));

}

int main(void)

{

     int* array = malloc(10 * sizeof(Int));

     set_one_bit_in_word(array, 177);

     printf("%d \n", get_one_bit_in_
word(array, 178));

     return 0;

}
```

Assuming that the size of an integer is 32 bits, the functions *set_one_bit_in_word* and *get_one_bit_in_word* set a bit and return the value of a bit in the word. In the *'main '* function, the code allocates an array of 10 integers. It then sets a bit in the word represented by *array[5]*, at the bit position 17. It then calls *'printf '* and tries to print the value of the bit position 18—which is undefined, since it was not set by the function *'set_one_bit_in_word '*. Therefore, the above code snippet suffers from the coding bug of using an uninitialised value in the call to *'printf '*. However, it is not easy to detect this issue.

Assume that the error detection tool maintains shadow memory information at word granularity. Hence, when the function *'set_one_bit_in_word '* was called, the tool updates the shadow memory corresponding to the memory location initialised in that function. Since the tracking is at word granularity, the tool records that the whole word (containing the bit that is being set) is initialised, when this is not correct. Therefore, the coding error in the next line, when the uninitialised bit value is passed to *printf*, cannot be caught by the tool. In order to detect this error, we need to maintain shadow memory information at bit precision—that is, we need to be able to track every bit location in our dynamically allocated memory chunks by memory shadowing. This, in turn, imposes considerable memory overheads on the application, in order to support shadow memory at such a fine granularity of tracking.

There is a trade-off involved between having the ability to track application memory locations at bit precision, versus the space overheads imposed by the shadow memory for maintaining such precision. Many of the runtime analysis tools for detecting memory errors use memory shadowing for detecting errors involving the use of undefined values in the application.

Some popular tools that use shadow values are:

a) Memcheck, based on Valgrind's Dynamic Binary Instrumentation (DBI) framework *(http://www.valgrind.org/)*
b) Rational Purify from IBM *(http://www.ibm.com/software/awdtools/purify/)*
c) PIN from Intel *(http://www.pintool.org/)*

While this discussion focused on using shadow memory to detect undefined values, there are a number of other uses of shadow memory, such as taint checking, race detection, etc. In next month's column, I will discuss different techniques that are used to implement shadow memory efficiently, as well as the different usage of shadow memory in runtime analysis tools.

There has been considerable research on dynamic memory error detection. One of the current state-of-the-art research tools in this area is Dr.memory, available at *http://dynamorio.org/drmemory.html*. If you are interested in learning more about it, read the in-depth introduction to this tool, available at *http://groups.csail.mit.edu/commit/papers/2011/bruening-cgo11-drmemory.pdf*.

My 'must-read book' for this month

This month's 'must-read book' suggestion comes from one of our readers, Natarajan, who recommends 'Programming Language Pragmatics' by Michael L. Scott (*http://www.cs.rochester.edu/~scott/pragmatics/*). He says that this book gives an excellent overview of parsing, grammar, automata theory and other key language constructs, in a manner easily understandable by the student, with lots of examples. One of the interesting aspects of this book is that it has a section on various programming models. This section covers functional languages, logical languages, scripting languages and concurrency in a concise manner. I agree that Scott's book is a good one to read; however, the caveat is that this is a book that requires considerable concentration to grasp the subtle language concepts, and hence should not be considered as a light afternoon read. However, it's definitely a must if you want to become a compiler developer. If you have a favourite programming book that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful, so I can mention it in the column. This would help many readers who want to improve their coding skills.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Till we meet again next month, happy programming, and here's wishing you the very best! END

By: Sandya Mannarswamy

The author is an expert in systems software, and works at Hewlett-Packard India. Her interests include compilers, virtualisation technologies and software development tools. If you are preparing for systems software interviews, you may find it useful to visit Sandya's website *http://www.techpathshala.com*, which offers practice interviews and training for systems software jobs, as well as her LinkedIn group tech-pathshala's system software training group at *http://www.linkedin.com/groups?home=&gid=3813966&trk=anet_ug_hm*.